OBSERVABILIDADE · AUTOCHECKER

Estudo das Etapas 3 e 5 do Cronograma

Cruzamento do código ao vivo (Azure Repos, branch `main`) com os work items do Azure Boards · org `automindautomacao` / projeto `AutoChecker`

Data: 16/06/2026 · Fonte: MCP Azure DevOps + código-fonte verificado

---

1. Visão geral

	Etapa 3 (item #3)	Etapa 5 (item #5)
Título	Limpar `PeriodicTask` órfã do Beat (`KeyError process_scheduled_notifications`)	Consolidar init do Sentry no helper `init_sentry` (+ `CeleryIntegration` no backend)
Onda / Camada / Esforço	Onda 0 · Backend/Worker · P	Onda 0 · Backend/Worker · P
Natureza	Código já limpo → falta **runtime/ops**	Pura **configuração de SDK** (código)
Work item no Azure	#53659 Done, #53660 / #53707 To Do	**#55421** To Do

O board do Azure **já tem tickets que batem exatamente com as duas etapas** — não é trabalho do zero, é fechar o que já está aberto. Feature-pai: **#40531 "Observabilidade"**.

2. Etapa 3 — `PeriodicTask` órfã do Beat

O que é

A task `process_scheduled_notifications` foi **comentada** no código, mas o agendador persiste o schedule **fora do código**. Sobra uma entrada órfã apontando para uma task que não existe mais → `KeyError` no dispatch do Beat. Era o **maior ruído do produto** (~1.640 ev — clusters BACKEND-1H / WORKER-1F).

Estado atual no repositório (confirmado ao vivo)

- `worker/apps/notifications/tasks/notification_tasks.py` — `process_scheduled_notifications` e `cleanup_old_notification_logs` estão **comentadas**. ✓ (foi o que *criou* a órfã)
- `worker/config/celery.py` — `beat_schedule` tem 8 entradas e **nenhuma** é a task removida. ✓
- `backend/config/celery.py` — **não define** `beat_schedule` e agora tem comentário explícito explicando o porquê (resultado do bug #53659). ✓

- **Onde a órfã realmente vive:** o Beat roda com `PersistentScheduler` → arquivo `/tmp/celerybeat-schedule` (shelve), persistido pelo volume `/tmp`. **Não** é o DB.

> ⚠ **Correção ao xlsx:** o cronograma assumia a órfã no **DB** (`django_celery_beat.PeriodicTask`). O código mostra que `django_celery_beat` está em `INSTALLED_APPS` mas **não é o scheduler em uso** — a órfã está no **arquivo shelve**. Verificar os dois por garantia.

Work items que tratam disso

ID	Tipo	Estado	O que cobre
#53659	Bug	Done	Limpar beat_schedule do backend + resetar celerybeat-schedule (lado-código, já feito)
#53707	Task	To Do	**Coração da etapa:** limpeza preventiva do celerybeat-schedule no deploy (expurgo runtime)
#53660	Task	To Do	Duplicata da #53659
#40384	Task	Done	"Ajustar tasks do Celery BEAT" (histórico)

Meios para resolver

1. **Diagnosticar onde a órfã está** — inspecionar `/tmp/celerybeat-schedule*` no container do beat **e** checar `PeriodicTask.objects.filter(task__icontains='process_scheduled_notifications')` no DB.
2. **Expurgar** (entrega real, #53707):
   - Se for o arquivo shelve (mais provável): parar o beat, `rm -f /tmp/celerybeat-schedule*`, subir de novo — recria a partir do `beat_schedule` do código.
   - Se houver linha no DB: **data migration idempotente** (`PeriodicTask… filter(task__endswith='process_scheduled_notifications').delete()`), versionada.
3. **Travar a fonte de verdade** — manter o file scheduler; **não** misturar com `DatabaseScheduler` (a mistura é o que gera órfãs).
4. **Validar** — sem mais `KeyError` no dispatch; queda dos clusters BACKEND-1H / WORKER-1F no Sentry (~1.640 → ~0). Teste de guarda: cada task do `beat_schedule` registrada em `app.tasks`.

> ⚠ **Runtime:** o passo 2 não versiona em código. Apagar arquivo de ambiente / rodar migration de delete deve ser **confirmado antes** de executar.

3. Etapa 5 — Consolidar o init do Sentry no `init_sentry`

O que é

Backend e worker fazem `sentry_sdk.init` **inline** e **divergem**. O helper `shared/observability/sentry.py::init_sentry()` existe mas **ninguém chama em produção** (só nos testes). A entrega é migrar os dois para o helper — **com cuidado**, porque cada init tem hoje coisas que o outro não tem.

Estado atual no repositório (confirmado ao vivo)

Local	Integrations	`before_send`
Backend `settings/base.py`	Django, Redis (sem Celery)	`_sentry_before_send` : `/health/` **+ filtro de Redis transiente** (`_is_redis_transient_event`)
Worker `config/celery.py:20-32`	Django, **Celery**, Redis	lambda: **só** `/health/` (sem scrubbing de breadcrumb)
Helper `shared/observability/sentry.py`	Django, Redis; **Celery só se** `service=="worker"`	`before_send_filter` : `/health/` **+ scrubbing de query nos breadcrumbs**

> ⚠ **Ponto crítico (REV 10/06, confirmado):** trocar só a chamada **não dá** `CeleryIntegration` **ao backend**, e o helper **não tem** o filtro de Redis transiente do backend. Migração ingênua = **regressão**.

Work items que tratam disso

ID	Tipo	Estado	O que cobre
#55421	Task	To Do	**É a Etapa 5:** "[Obs. Onda 0 / E5] Consolidar init do Sentry no helper init_sentry (backend + worker)"
#40531	Feature	New	Épico-pai "Observabilidade"
#48859 / #48860	PBI / Task	Done	Setup inicial do Sentry (histórico)

Meios para resolver

1. **Alterar o helper primeiro** para não perder nada:
   - Dar `CeleryIntegration` ao backend também, ou parametrizar as integrations por serviço.
   - Levar o `_is_redis_transient_event` (hoje só no backend) para dentro do `before_send_filter` do helper.
2. **Migrar as duas chamadas** para `init_sentry("backend")` / `init_sentry("worker")`. Ganho de brinde: o worker passa a ter o scrubbing de breadcrumb que hoje não tem.
3. **Fixar** `send_default_pii=False` explicitamente (cruza com o item 7 — PII/LGPD).

4. **Teste de guarda** afirmando o conjunto de integrations esperado por serviço (pega regressão em CI).

4. Bug colateral (atravessa a Etapa 3)

No `worker/apps/notifications/tasks/notification_tasks.py`, a versão **local** de `retry_failed_notifications` chama `process_notification_task.delay(notification.id)` com **1 argumento**. A assinatura canônica em `shared/tasks/notification_tasks.py` exige **3**: `(notification_log_id, tenant_schema_name, tenant_id)` → `TypeError` em runtime se invocada. A versão do `shared` já foi reescrita multi-tenant (correta).

- #53657 Done — refatoração multi-tenant no `shared` (3 args, feito).
- #53705 To Do — **fix do colateral:** "Remover retry_failed_notifications antiga do worker (continuação de 53657)". Remover a versão local e reexportar a do `shared` (como o backend já faz).

De passagem: o `include` do `worker/config/celery.py` lista `shared.tasks.notification_tasks` duas vezes — limpeza trivial.

5. Resumo — o que falta fechar no board

Etapa	Ticket aberto	O que ainda falta
3	#53707 To Do	Expurgo runtime do `/tmp/celerybeat-schedule` + prevenção no deploy + teste de guarda
3 (colateral)	#53705 To Do	Remover `retry_failed_notifications` bugada do worker
5	#55421 To Do	Alterar helper (Celery no backend + filtro Redis) → migrar os 2 inits → teste de integrations

A **Etapa 3** é majoritariamente **ops** (rodar no ambiente, confirmar antes); a **Etapa 5** é **100% código** e de baixo risco se o helper for ajustado antes da migração.

Gerado em 16/06/2026 · dados ao vivo via MCP Azure DevOps (Repos + Boards) · branch `main` · org automindautomacao / projeto AutoChecker. Wiki do projeto: sem acesso (401).